Project Final Report

Date	29 September 2026
Team ID	SWTID-2026-1394
Team Leader	Rafeena.A
Team members	Abinesh.S.M,Mohamed Najeeb.M,Samuel.S
Project Name	Import Data using Transform Maps (Spreadsheet)

1. INTRODUCTION

1.1 Project Overview

This project is an automation and data management system designed to properly map and import raw data from spreadsheets (CSV/Excel) into a database. It helps users to create flexible transformation rules for data conversition

1.2 Purpose

- To avoid manual data entry errors.

- To connect columns in Excel or CSV files with database fields through Transform Maps.

- To automate the data migration process and save time when handling large volumes of data.

2. IDEATION PHASE

2.1 Problem Statement

When separately restructuring and inserting spreadsheet data received from external sources into a database, problems such as field mismatch, formatting errors, and data loss occur. The lack of an automated mapping solution to avoid this is a major obstacle.

2.2 Empathy Map Canvas

- *Says:* "It is very difficult to bring the data in the Excel sheet into the database at once."

- *Thinks:* "If there was a custom transform map, we could easily convert the data to our required format."

- *Does:* Manually copying and pasting row by row or writing custom scripts.

- *Feels:* Fear of data validation errors and feeling that a lot of valuable time is being wasted.

2.3 Brainstorming

- Introducing a drag-and-drop file upload interface.

- Creating a 'Transform Map' logic to match spreadsheet columns and database schemas.

- Filtering and displaying rows with errors separately.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

1. *Discovery:* User uploads their spreadsheet to the app.

2. *Mapping:* User maps spreadsheet columns to database fields (Transform Map creation).

3. *Validation & Import:* System validates the data and sends it to the database.

4. *Completion:* User views the Success/Failure report and error logs.

3.2 Solution Requirement

- *Functional Requirements:* File upload (CSV/Xlsx), Column mapping interface, Data validation, Transform rules engine.

- *Non-Functional Requirements:* Security, fast file processing, scalable database integration.

3.3 Data Flow Diagram

User ->Upload Spreadsheet ->Backend File Parser ->Column Mapper UI ->Transform Engine ->MongoDB Database.

3.4 Technology Stack

- *Frontend:* React.js, Tailwind CSS

- *Backend:* Node.js, Express.js

- *Database:* MongoDB

- *Libraries:* Multer, SheetJS (xlsx)

4. PROJECT DESIGN

4.1Problem Solution Fit

The main highlight of this project is that instead of directly connecting the spreadsheet to the database, it sanitizes and loads the data by keeping an intermediate Transform Map layer.

4.2Proposed Solution

A unified web dashboard. It provides file upload, mapping configuration, preview, and execution status in one place for the user.

4.3 Solution Architecture

The client-side React app communicates with the Node.js backend through REST APIs. The parser module in the backend reads the file, applies Transform rules, and saves it to MongoDB.

5. PROJECT PLANNING&SCHEDULING

5.1Project Planning

- *Phase 1 (Week 1-2):* Requirements gathering, UI wireframes, Database schema design.

- *Phase 2 (Week 3-4):* Backend file parsing logic and REST API creation.

- *Phase 3 (Week 5-6):* Frontend-Backend integration, Transform Map UI implementation.

- *Phase 4 (Week 7):* Testing, Bug fixing, Documentation preparation.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1Performance Testing

- Processing speed was checked by uploading a CSV file with 10,000 rows.

- Memory usage and server response time were tested and optimized.

7. RESULTS

7.1Output Screenshots*

- [File Upload Screen UI Screenshot]

- [Transform Map Column Matching UI Screenshot]

- [Import Summary&Logs Dashboard Screenshot]

8. ADVANTAGES&DISADVANTAGES

- *Advantages:*

  - Automated data mapping process.

  - Time-saving and error reduction.

  - User-friendly interface for non-technical users.

- *Disadvantages:*

  - Processing extremely large files (>50MB) may require more server memory.

9. CONCLUSION

This "Import Data using Transform Maps (Spreadsheet)" project has greatly simplified the data migration and spreadsheet integration process. It has been developed as a good scalable solution using the MERN stack.

10. FUTURE SCOPE

- AI-powered automatic column matching (smart mapping).

- Support for SQL databases and cloud storage (AWS S3) integration.

- Background queue processing for heavy multi-user workloads.

11. APPENDIX

- *Source Code:* [Add GitHub Repository Link here]

- *Dataset Link:* [Add Sample CSV/Excel dataset link here]

- *GitHub & Project Demo Link:* [Live Demo Link / GitHub Link]